

PRD & Walkthrough — Pembangunan Model TinyML Deteksi Aritmia

Implementasi Sistem Monitoring Detak Jantung Kontinu Berbasis IoT Edge
Computing untuk Deteksi Dini Aritmia

Muhammad Rafi Rizaldi (3123600001) — D4 PENS

Fase 0 → PoC (software-only, tanpa hardware)

Contents

0. Tentang Dokumen Ini	3
Filosofi dokumen: manual, bukan vibe coding	3
Definisi PoC (target akhir dokumen ini)	3
Yang TIDAK dicakup dokumen ini	3
1. Ruang Lingkup & Prinsip Non-Negosiasi	3
Dua jalur validasi (jangan pernah dicampur)	3
Prinsip disiplin yang harus dipegang	4
2. Glosarium Singkat	4
3. Struktur Repo yang Disarankan	4
4. Prasyarat & Konvensi	5
Environment	5
Konvensi angka (catat, ini decision point yang di-lock)	5
FASE 0 — Environment & Data	6
Tujuan	6
Kenapa	6
Yang dibangun	6
Decision point	6
Jebakan	7
Definition of Done	7
Cara verifikasi	7
FASE 1 — Preprocessing (Golden Reference)	7
Tujuan	7
Kenapa (baca pelan-pelan — ini paling penting)	7
Yang dibangun — src/preprocessing.py	7
Decision point	9
Jebakan	9
Definition of Done	9
Cara verifikasi	9
FASE 2 — Ekstraksi Beat, Label, & Fitur RR	9
Tujuan	9
Kenapa	10
Keputusan yang sudah dikunci (dari diskusi sebelumnya)	10

Yang dibangun	10
Definition of Done	11
Cara verifikasi	11
FASE 3 — Split Inter-Patient DS1/DS2	11
Tujuan	11
Kenapa	11
Yang dibangun — daftar record (de Chazal 2004)	11
Jebakan	12
Definition of Done	12
Cara verifikasi	12
FASE 4 — Arsitektur Model Hybrid	12
Tujuan	12
Kenapa	12
Yang dibangun — src/model.py	13
Decision point	13
Definition of Done	13
Cara verifikasi	13
FASE 5 — Training + Penanganan Imbalance	13
Tujuan	13
Kenapa	14
Yang dibangun — src/train.py	14
Decision point	14
Jebakan	14
Definition of Done	14
Cara verifikasi	14
FASE 6 — Evaluasi Float32 di DS2	15
Tujuan	15
Kenapa	15
Yang dibangun — src/evaluate.py	15
Decision point	15
Jebakan	15
Definition of Done	15
Cara verifikasi	15
FASE 7 — Kuantisasi INT8 + Tabel Delta	16
Tujuan	16
Kenapa	16
Yang dibangun — src/quantize.py	16
Decision point	17
Jebakan	17
Definition of Done	17
Cara verifikasi	17
FASE 8 — PoC Selesai: Checklist Final	17
LAMPIRAN A — Urutan Eksekusi Cepat	18
LAMPIRAN B — Peta “Why A not B” (amunisi sidang)	18
LAMPIRAN C — Yang Menunggu Hardware (di luar PoC)	18

0. Tentang Dokumen Ini

Dokumen ini adalah **PRD (Product Requirements Document)** sekaligus **walkthrough belajar** untuk membangun *seluruh jalur model* PA lo — dari nol sampai **Proof of Concept (PoC)** — tanpa perlu satu komponen hardware pun. Semua yang ada di sini bisa dikerjakan di laptop biasa selama nunggu alat datang.

Filosofi dokumen: manual, bukan vibe coding

Dokumen ini **sengaja tidak ngasih solusi jadi yang tinggal copy-paste**. Untuk tiap fase lo dapat:

- **Tujuan** — apa yang harus jadi di akhir fase.
- **Kenapa** — rasional teknis + kaitannya ke klaim ilmiah PA lo (ini yang bakal ditanya penguji).
- **Yang dibangun** — kontrak fungsi (signature + docstring), pseudocode, dan konstanta/config yang boleh langsung dipakai. Algoritma inti (Pan-Tompkins, biquad, training loop) ditinggalin sebagai **stub + TODO** biar lo yang nulis.
- **Decision point** — keputusan yang harus lo ambil sadar dan lo catat (bukan default diam-diam).
- **Jebakan** — bug/kesalahan metodologi yang umum dan bakal diserang penguji.
- **Definition of Done (DoD)** — checklist “fase ini selesai kalau...”.
- **Cara verifikasi** — cara ngetes bahwa hasil lo benar.

Kalau lo cuma nyalin kode tanpa ngerti kenapa, lo bakal keok di sidang. Jadi tiap kali ketemu blok “Kenapa” dan “Jebakan”, baca pelan-pelan.

Definisi PoC (target akhir dokumen ini)

PoC dianggap **selesai** kalau lo punya:

1. Pipeline preprocessing offline yang **deterministik dan bisa direproduksi** (input record MIT-BIH → keluar window ter-segmentasi + fitur RR + label biner).
2. Model hybrid dua-cabang terlatih dalam format **float32**, dievaluasi pada **DS2 (inter-patient)** dengan confusion matrix + accuracy/precision/recall/F1.
3. Model yang sama dikuantisasi ke **INT8** dan dievaluasi ulang pada DS2 identik, dengan **tabel delta** tiap metrik float32 vs INT8.
4. Jumlah parameter model terverifikasi ~6.000 dan ukuran .tflite INT8 terverifikasi < 20 KB.

Kalau keempat itu ada, klaim ilmiah utama PA lo (inferensi aritmia on-device via TinyML) sudah **terbukti layak secara software** — tinggal nunggu hardware buat benchmarking latensi/RAM/daya.

Yang TIDAK dicakup dokumen ini

- Firmware C/C++ ESP32-S3 (butuh hardware + TFLite Micro build).
- Benchmarking on-device: latensi per-detak, tensor arena RAM, konsumsi daya INA219.
- Pipeline resilience testing (PSRAM ring buffer, MQTT offloading, ThingsBoard).
- Pengujian fungsional dengan relawan sehat.

Semua itu adalah jalur terpisah yang butuh alat. Dokumen ini fokus **100% ke jalur model** karena itulah yang bisa dikerjakan sekarang dan itulah klaim ilmiah primer lo.

1. Ruang Lingkup & Prinsip Non-Negosiasi

Dua jalur validasi (jangan pernah dicampur)

PA lo punya dua jalur pengujian yang **secara ilmiah harus dipisah tegas**:

	Jalur Model (dokumen ini)	Jalur Pipeline (butuh hardware)
Tujuan	Klaim ilmiah: akurasi klasifikasi	Fungsional: alur end-to-end jalan
Data	MIT-BIH DS2, inter-patient	Relawan sehat (diri/teman)
Metrik	accuracy/F1/recall Aritmia	sinyal→filter→R-peak→inferensi→dashboard nyambung
“Diagnosis benar/tidak?”	Relevan (ini inti klaim)	Tidak relevan (cuma cek eksekusi)

Prinsip: metrik jalur model (MIT-BIH, inter-patient, ilmiah) **tidak boleh** dicampur atau digeneralisasi dari data relawan. Jangan pernah bilang “akurasi X%” berdasarkan data relawan sehat — itu bukan validasi klinis.

Prinsip disiplin yang harus dipegang

- **Binary classification.** Task = Normal vs Aritmia. Bukan 5-kelas AAMI. Alasan: di bawah split inter-patient, kelas F (fusi) dan Q (unclassifiable) punya sampel sangat sedikit → recall mendekati nol → metrik menyesatkan. Binary framing adalah pilihan yang benar untuk scope ini, dan itu harus bisa lo pertahankan di sidang.
- **Inter-patient split, bukan intra-patient.** Pasien di data latih ≠ pasien di data uji. Ini yang bikin metrik lo mencerminkan generalisasi ke pasien baru, bukan hafalan. Ini pembeda kualitas metodologi dari banyak paper “akurasi 99%” yang diam-diam pakai intra-patient.
- **Zero data leakage.** Statistik apapun yang di-fit (kalau ada normalisasi global fitur RR, dsb.) di-fit **hanya di DS1**, lalu diterapkan ke DS2. z-score morfologi aman karena per-window (lihat Fase 1).
- **Reproducible.** Seed di-lock, daftar record di-hardcode di config, tidak ada randomness yang gak tercatat.

2. Glosarium Singkat

- **Beat / detak** — satu siklus jantung, diwakili satu R-peak + window di sekitarnya.
- **Window** — potongan sinyal 250 sampel di sekitar satu R-peak (input cabang morfologi).
- **Fitur RR** — 3 angka turunan interval antar R-peak (input cabang ritme): RR_prev, rasio RR, ΔRR.
- **DS1 / DS2** — partisi record MIT-BIH untuk latih / uji, skema de Chazal (inter-patient).
- **PTQ** — Post-Training Quantization: kuantisasi setelah model dilatih (bukan saat latih).
- **Representative dataset** — sampel data kecil buat kalibrasi rentang kuantisasi INT8.
- **Golden reference** — implementasi preprocessing Python yang nanti harus **ditiru identik** di firmware C. Kalau Python dan C beda, akurasi device ≠ akurasi lab.

3. Struktur Repo yang Disarankan

Bikin repo **terpisah** dari repo LaTeX PA. Struktur:

```

pa-model/
├── README.md
├── requirements.txt
├── config.py                # semua konstanta (freq, window, record lists, seed)

```

```

├── data/
│   ├── raw/           # MIT-BIH hasil download (jangan di-commit; .gitignore)
│   └── processed/     # array beat/label/RR hasil Fase 2 (.npz)
├── src/
│   ├── io_mitdb.py     # loader wfdb (Fase 0)
│   ├── preprocessing.py # biquad + Pan-Tompkins + segmentasi + z-score (Fase 1)
│   ├── features_rr.py  # ekstraksi fitur RR (Fase 2)
│   ├── dataset.py      # rakit X_morph, X_rr, y + split DS1/DS2 (Fase 3)
│   ├── model.py        # arsitektur hybrid (Fase 4)
│   ├── train.py        # training + class weight (Fase 5)
│   ├── evaluate.py     # metrik + confusion matrix (Fase 6)
│   └── quantize.py     # PTQ INT8 + delta (Fase 7)
├── notebooks/
│   └── sanity_checks.ipynb # plot buat verifikasi tiap fase
├── artifacts/
│   ├── model_fp32.keras
│   ├── model_int8.tflite
│   └── metrics/         # confusion matrix, tabel delta (.csv/.png)
└── tests/
    └── test_preprocessing.py

```

Kenapa dipisah modul per fase: biar tiap fase punya DoD sendiri dan bisa diverifikasi independen sebelum lanjut. Jangan bikin satu file main.py raksasa.

4. Prasyarat & Konvensi

Environment

- Python 3.10–3.11 (hindari yang terlalu baru biar TFLite converter stabil).
- Virtual environment wajib (python -m venv .venv).
- requirements.txt minimal:

```

numpy
scipy
wfdb
tensorflow           # versi yang ada tf.lite converter; 2.15–2.17 aman
scikit-learn         # metrik + class_weight
matplotlib            # verifikasi visual

```

Konvensi angka (catat, ini decision point yang di-lock)

Isi config.py dengan konstanta yang **sesuai dokumen PA lo**:

```

# config.py – SEMUA KONSTANTA DI SATU TEMPAT

FS = 360                # Hz, sampling MIT-BIH (dan device disamakan)
BANDPASS_LOW = 0.5      # Hz, cutoff bawah Butterworth
BANDPASS_HIGH = 40.0    # Hz, cutoff atas Butterworth
BANDPASS_ORDER = 4      # orde 2–4; kunci satu nilai, jangan ganti diam-diam

WIN_PRE = 90            # sampel sebelum R-peak
WIN_POST = 160          # sampel sesudah R-peak
WIN_LEN = WIN_PRE + WIN_POST # = 250 (WAJIB 250, bukan 500)

CHANNEL = "MLII"        # kanal utama MIT-BIH
SEED = 42               # lock semua randomness ke sini

# Peta anotasi → label biner (Tabel 3.1 di dokumen PA)
NORMAL_SYMBOLS = {"N"}  # hanya sinus normal = Normal

```

```

ARRHYTHMIA_SYMBOLS = {"V", "S", "F", "Q"} # sisanya = Aritmia
# catatan: simbol MIT-BIH mentah kadang pakai N,L,R,e,j,A,a,J,S,V,E,F,/,f,Q,...
# lo perlu MAPPING dari simbol mentah ke {N, V, S, F, Q} dulu (lihat Fase 2)

```

Jebakan window 250: label “ ± 250 sampel di R” itu keliru — artinya jadi 500 total. Yang benar **250 total** (90 pre + 160 post). Konsisten dengan yang bakal lo perbaiki di dokumen (Gambar 3.3).

FASE 0 — Environment & Data

Tujuan

Punya MIT-BIH ter-download lokal + loader yang bisa baca satu record jadi (sinyal MLII, lokasi R-peak anotasi, simbol anotasi).

Kenapa

Semua fase berikutnya makan output loader ini. Salah baca kanal atau salah baca anotasi = seluruh pipeline salah diam-diam.

Yang dibangun

0.1 Download dataset

wfdb bisa nge-pull MIT-BIH langsung dari PhysioNet:

```

# scripts sekali jalan
import wfdb
wfdb.dl_database("mitdb", dl_dir="data/raw/mitdb")

```

0.2 Loader — src/io_mitdb.py

Implement sendiri (stub):

```

import wfdb
import numpy as np
from config import CHANNEL

def load_record(record_id: str, raw_dir: str = "data/raw/mitdb"):
    """
    Baca satu record MIT-BIH.

    Returns:
        signal (np.ndarray, shape [N]): sinyal kanal MLII, float.
        r_locations (np.ndarray, shape [M]): indeks sampel R-peak (dari anotasi).
        symbols (list[str], len M): simbol anotasi per beat (mentah).
        fs (int): sampling frequency (harus 360).

    TODO:
    - pakai wfdb.rdrecord(...) untuk sinyal; pilih kolom kanal MLII
      (hati-hati: sebagian record kolom MLII bukan index 0).
    - pakai wfdb.rdann(..., extension="atr") untuk anotasi.
    - filter anotasi: buang simbol non-beat (mis. '+', '~', '|', 'x')
      yang menandai ritme/artefak, bukan detak. Simpan hanya beat annotation.
    - assert fs == 360.
    """
    raise NotImplementedError

```

Decision point

- **Kanal MLII bukan selalu channel 0.** Beberapa record (mis. 114) urutan kanalnya kebalik. Baca record.sig_name dan cari index "MLII" secara eksplisit. Catat record mana yang anomali.

Jebakan

- **Anotasi MIT-BIH bercampur simbol non-beat.** Simbol seperti + (rhythm change), ~ (signal quality), | (isolated QRS-like), x, !, [,] **bukan detak.** Kalau ikut dibawa, RR interval lo kacau. Filter dulu ke daftar simbol beat yang valid.
- **Jangan pakai record paced** (102, 104, 107, 217) sama sekali — di-exclude di Fase 3, tapi jangan sampai ikut di eksplorasi awal terus lupa dibuang.

Definition of Done

- ☐ `load_record("100")` mengembalikan sinyal panjang ~650.000 sampel (≈ 30 menit $\times$ 360 Hz), plus ~2.200-an lokasi R-peak.
- ☐ `fs == 360` ter-assert.
- ☐ Simbol non-beat sudah terbuang.

Cara verifikasi

Plot 10 detik pertama record 100, overlay garis vertikal di tiap `r_locations` yang jatuh di rentang itu. Garis harus mendarat pas di puncak R. Kalau meleset → loader/anotasi salah baca.

FASE 1 — Preprocessing (Golden Reference)

Tujuan

Rantai preprocessing yang **kausal dan streaming-friendly**: Butterworth bandpass 0,5–40 Hz → (Pan-Tompkins untuk deteksi R-peak) → segmentasi window 250 → z-score per window. Ini jadi **golden reference** yang nanti ditiru di C.

Kenapa (baca pelan-pelan — ini paling penting)

Preprocessing ini akan **dijalankan ulang di ESP32 dalam C**. Kalau versi Python lo dan versi C beda perilaku, model yang akurat di lab jadi ngaco di device — dan lo gak bakal tahu kenapa. Maka aturan besinya: **apa yang mustahil dijalankan real-time di MCU, jangan dipakai di training.**

JEBAKAN #1 (paling mematikan): jangan pakai `scipy.signal.filtfilt`

`filtfilt` itu **zero-phase / non-causal** — dia baca sinyal maju-mundur. Enak buat plot karena gak ada geseran fase, TAPI **mustahil** dijalankan sampel-demi-sampel di MCU (butuh seluruh sinyal masa depan). Kalau lo latih model pakai `filtfilt` lalu deploy biquad kausal di device, ada **train/deploy mismatch** yang menurunkan akurasi diam-diam.

Yang benar: filter **kausal** via `scipy.signal.sosfilt` (bukan `sosfiltfilt`), diimplementasi sebagai **second-order sections (SOS/biquad)**. Dokumen PA lo sudah nulis “kausal, biquad, sampel demi sampel” — tinggal implementasinya jangan khianat.

Yang dibangun — `src/preprocessing.py`

1.1 Butterworth bandpass kausal

```
import numpy as np
from scipy.signal import butter, sosfilt
from config import FS, BANDPASS_LOW, BANDPASS_HIGH, BANDPASS_ORDER

def design_bandpass_sos():
    """
    Rancang Butterworth bandpass sebagai SOS (biquad).
```

```

TODO: pakai butter(order, [low, high], btype="bandpass",
    fs=FS, output="sos"). Kembalikan array SOS.
Catatan: 'order' di butter untuk bandpass menghasilkan 2*order pole;
pilih BANDPASS_ORDER supaya jumlah biquad masuk akal untuk MCU (2-4 section).
"""
raise NotImplementedError

def apply_bandpass(signal: np.ndarray, sos) -> np.ndarray:
    """
    Filter KAUSAL. WAJIB sosfilt (bukan sosfiltfilt).
    TODO: return sosfilt(sos, signal).
    """
    raise NotImplementedError

```

1.2 Pan-Tompkins (deteksi R-peak) — stub

Ini algoritma yang lo implement sendiri dari paper Pan & Tompkins 1985. Tahapan (jangan pakai library jadi — inti belajarnya di sini):

```

def pan_tompkins_detect(signal: np.ndarray, fs: int = FS) -> np.ndarray:
    """
    Deteksi lokasi R-peak dari sinyal yang sudah di-bandpass.

    Tahapan (implement berurutan, semua kausal):
    1. Bandpass 5-15 Hz (boleh reuse struktur biquad; ini bandpass KEDUA
       khusus penonjolan QRS, beda dari bandpass 0.5-40 di 1.1).
    2. Turunan (derivative filter 5-tap khas Pan-Tompkins).
    3. Kuadratkan (squaring) — nonlinear, menonjolkan energi QRS.
    4. Moving Window Integration (window ~150 ms = ~54 sampel @360Hz).
    5. Threshold adaptif (SPKI/NPKI) + periode refraktori ~200 ms
       untuk cegah deteksi ganda.

    Returns: np.ndarray indeks sampel R-peak terdeteksi.

    TODO: implement penuh. Refractory period WAJIB, kalau tidak
    satu QRS kedeteksi 2-3 kali.
    """
    raise NotImplementedError

```

Catatan penting soal peran Pan-Tompkins di jalur model: untuk *melatih model*, segmentasi window pakai **R-peak dari ANOTASI MIT-BIH**, bukan dari Pan-Tompkins lo (lihat Fase 2, keputusan sudah dikunci). Jadi kenapa tetap implement Pan-Tompkins sekarang? 1. Ini bagian golden reference yang harus ada di device. 2. Lo bisa bikin **benchmark deteksi opsional**: jalankan Pan-Tompkins di record MIT-BIH, cocokkan hasilnya dengan anotasi (toleransi ± 150 ms), hitung sensitivity & PPV deteksi. Angka ini jawaban buat penguji yang nanya “seberapa akurat R-peak detector on-device lo?” — dan itu **terpisah** dari klaim klasifikasi.

1.3 Segmentasi window

```

from config import WIN_PRE, WIN_POST, WIN_LEN

def segment_beats(signal: np.ndarray, r_locations: np.ndarray):
    """
    Potong window 250 sampel (90 pre + 160 post) di tiap R-peak.

    Returns: np.ndarray shape [K, WIN_LEN] (K = jumlah beat valid).

    TODO:
    - untuk tiap r di r_locations: window = signal[r-WIN_PRE : r+WIN_POST].
    - BUANG beat yang windownya keluar batas sinyal (r terlalu awal/akhir).
      Catat berapa yang dibuang (biasanya cuma beat pertama & terakhir record).
    - pastikan setiap window PERSIS WIN_LEN; kalau tidak, jangan di-pad
      diam-diam, buang saja (konsistensi > jumlah data).
    """
    raise NotImplementedError

```

1.4 Normalisasi z-score per window

```
def zscore_per_window(windows: np.ndarray, eps: float = 1e-8) -> np.ndarray:
    """
    z-score MASING-MASING window: (x - mean_window) / (std_window + eps).

    Kenapa per-window (bukan global): model fokus ke BENTUK sinyal,
    kebal terhadap beda amplitudo antar-perangkat & antar-pasien.
    Bonus metodologis: karena per-window, TIDAK ADA statistik global yang
    di-fit → tidak ada jalur kebocoran data dari test ke train di sini.

    TODO: hitung mean & std per baris (axis=1, keepdims=True).
    """
    raise NotImplementedError
```

Decision point

- **Orde Butterworth** (2 vs 4): kunci satu, catat di config, konsisten Python↔C. Orde 4 = lebih tajam tapi 2 biquad; orde 2 = lebih ringan.
- **Panjang MWI window Pan-Tompkins** (~150 ms): angka klasik, tapi catat nilai sampelnya eksplisit.

Jebakan

- `filtfilt/sosfiltfilt` → **haram** (lihat JEBAKAN #1).
- Segmentasi keluar batas array → `IndexError` atau window kepotong. Selalu cek bound.
- Double-detection R-peak karena lupa refractory period.

Definition of Done

- ☐ `apply_bandpass` pakai `sosfilt` (kausal), sudah dicek bukan `filtfilt`.
- ☐ Pan-Tompkins jalan di record 100, hasil deteksinya masuk akal (jumlahnya dekat jumlah anotasi).
- ☐ `segment_beats` menghasilkan window persis 250 sampel, beat out-of-bound terbuang & tercatat.
- ☐ z-score per-window: tiap window punya $\text{mean} \approx 0$, $\text{std} \approx 1$.

Cara verifikasi

1. **Filter:** plot 5 detik sinyal mentah vs hasil bandpass. Baseline wander (naik-turun lambat) harus hilang; bentuk QRS tetap.
 2. **Pan-Tompkins:** overlay R-peak terdeteksi vs anotasi di 10 detik. Hitung berapa cocok dalam ± 150 ms. Target sensitivity tinggi ($>99\%$ di record bersih macam 100).
 3. **Window:** plot 5 window Normal ditumpuk — harus mirip bentuk. Plot 5 window V (PVC) — harus jelas beda (lebar, tanpa P).
 4. **z-score:** `windows_norm.mean(axis=1)  $\approx 0$` , `windows_norm.std(axis=1)  $\approx 1$` .
-

FASE 2 — Ekstraksi Beat, Label, & Fitur RR

Tujuan

Dari tiap record: hasilkan (`window_ternormalisasi`, `fitur_RR`, `label_biner`) per beat.

Kenapa

Ini yang mengubah sinyal mentah jadi **contoh latih**. Di sini juga keputusan besar “R-peak dari anotasi vs Pan-Tompkins” dipakai.

Keputusan yang sudah dikunci (dari diskusi sebelumnya)

Segmentasi window untuk training pakai R-peak dari ANOTASI MIT-BIH. Alasan:

- Jalur model itu soal **klasifikasi diberi sebuah beat**, bukan soal deteksi. Kualitas deteksi = concern jalur pipeline/device, terpisah.
- Pakai anotasi = label bersih, tidak tercemar error detektor → metrik klasifikasi lo murni mengukur kemampuan model, bukan kemampuan Pan-Tompkins.
- Ini menjaga **pemisahan dua jalur validasi** tetap bersih (prinsip inti PA lo).

Siapkan jawaban penguji: “Apakah Pan-Tompkins on-device bikin segmentation drift vs anotasi, dan seberapa pengaruhnya?” → Jawab: diukur di benchmark deteksi terpisah (Fase 1.2), bukan mencemari klaim klasifikasi. Drift $\pm$ beberapa sampel diserap oleh window 250 yang cukup lebar.

Yang dibangun

2.1 Mapping simbol mentah → kelas → label biner — src/features_rr.py / dataset.py

MIT-BIH pakai banyak simbol. Lo perlu dua tahap mapping:

```
# Tahap 1: simbol mentah MIT-BIH → superclass {N, S, V, F, Q}
# (mengikuti konvensi AAMI; ini yang dipakai de Chazal)
```

```
AAMI_MAP = {
    # N (normal & bundle branch blocks)
    "N": "N", "L": "N", "R": "N", "e": "N", "j": "N",
    # S (supraventricular ectopic)
    "A": "S", "a": "S", "J": "S", "S": "S",
    # V (ventricular ectopic)
    "V": "V", "E": "V",
    # F (fusion)
    "F": "F",
    # Q (unknown/paced/unclassifiable)
    "/": "Q", "f": "Q", "Q": "Q",
}
```

```
def to_binary_label(aami_class: str) -> int:
    """
    N → 0 (Normal); {S,V,F,Q} → 1 (Aritmia). Sesuai Tabel 3.1.
    TODO.
    """
    raise NotImplementedError
```

Jebakan: simbol L dan R (bundle branch block) dalam konvensi AAMI masuk **N**, bukan aritmia. Ini beda dari intuisi awam. Catat keputusan ini dan konsisten — penguji bisa nanya “kenapa LBBB dianggap Normal?”. Jawaban: mengikuti pengelompokan AAMI/de Chazal supaya sebanding dengan literatur inter-patient. (Kalau advisor lo mau LBBB/RBBB masuk Aritmia, itu decision point yang harus di-lock eksplisit dan dicatat menyimpang dari AAMI.)

2.2 Fitur RR (3 dimensi)

```
def compute_rr_features(r_locations: np.ndarray, fs: int = FS):
    """
    Untuk tiap beat i, hitung 3 fitur ritme dari lokasi R-peak (anotasi):
    - RR_prev = (r[i] - r[i-1]) / fs          # interval ke beat sebelumnya (detik)
    - RR_ratio = RR_prev / RR_local_avg       # konteks ritme lokal
    - dRR      = RR_prev - RR_prev_sebelumnya # perubahan interval

    RR_local_avg: rata-rata RR pada jendela ~10 beat sekitarnya
```

(definisikan tepat & catat – ini decision point).

Returns: np.ndarray shape [M, 3]. Beat pertama tiap record TIDAK punya RR_prev → buang beat itu (konsisten dgn segmentasi).

TODO: hati-hati edge case awal record.

"""

raise NotImplementedError

2.3 Normalisasi fitur RR — AWAS KEBOCORAN DATA

Fitur RR dalam detik itu skalanya beda jauh dari window ter-z-score. Perlu diskalakan. **TAPI:**

JEBAKAN #2 (kebocoran data): kalau lo normalisasi fitur RR pakai mean/std global, **statistik itu WAJIB dihitung hanya dari DS1**, lalu diterapkan ke DS2. Kalau lo hitung mean/std dari gabungan DS1+DS2, itu **data leakage** — test set “membocorkan” info ke normalisasi, metrik jadi optimistis palsu, dan penguji yang paham langsung serang.

Aman-nya: pakai normalisasi yang **tidak butuh statistik global** (mis. RR dalam detik sudah $O(1)$ di rentang ~ 0.3 – 1.5 ; RR_ratio sudah ternormalisasi by design; dRR kecil). Kalau tetap mau standardisasi, fit scaler **di DS1 saja**.

Definition of Done

- Tiap record menghasilkan array sejajar: windows [K,250], rr [K,3], labels [K].
- Panjang ketiganya sama (K identik) — cek `assert len==len==len`.
- Mapping AAMI terdokumentasi, LBBB/RBBB→N tercatat sebagai keputusan sadar.
- Distribusi label dicetak per record (berapa Normal vs Aritmia).

Cara verifikasi

- Cetak distribusi kelas global: harus **sangat timpang** (Normal dominan). Ini normal untuk MIT-BIH dan justru alasan Fase 5 pakai class weighting.
- Ambil 3 beat berlabel Aritmia yang simbol aslinya V, plot windownya — harus terlihat morfologi PVC (QRS lebar).

FASE 3 — Split Inter-Patient DS1/DS2

Tujuan

Rakit dataset final terpisah **per pasien**: latih dari DS1, uji dari DS2, tanpa overlap subjek.

Kenapa

Ini **jantung klaim metodologi** lo. Split per-beat acak (intra-patient) bikin beat pasien yang sama nyebar di train & test → model “kenal” pasiennya → akurasi meroket palsu. Inter-patient (per-record/per-pasien) = uji jujur generalisasi ke orang baru.

Yang dibangun — daftar record (de Chazal 2004)

```
# config.py (lanjutan) – VERIFIKASI silang dgn de_chazal_2004 di biblio lo
DS1 = [101, 106, 108, 109, 112, 114, 115, 116, 118, 119, 122, 124,
        201, 203, 205, 207, 208, 209, 215, 220, 223, 230] # latih (22)
DS2 = [100, 103, 105, 111, 113, 117, 121, 123, 200, 202, 210, 212,
        213, 214, 219, 221, 222, 228, 231, 232, 233, 234] # uji (22)
PACED_EXCLUDED = [102, 104, 107, 217] # record ber-pacemaker, dibuang total
```

Decision point / verifikasi wajib: daftar di atas adalah split de Chazal standar (22+22, 4 paced dibuang = 48 record). Karena de_chazal_2004 ada di bibliografi lo, **cocokkan angka ini langsung ke papernya** sebelum eksekusi — ini gated decision, jangan asal percaya. DS1 sudah cocok dengan yang ada di file proyek lo; DS2 tetap cross-check.

Perakitan

```
def build_split(processed_by_record: dict):
    """
    Gabung semua beat DS1 → (X_morph_train, X_rr_train, y_train).
    Gabung semua beat DS2 → (X_morph_test, X_rr_test, y_test).
    Record di PACED_EXCLUDED tidak ikut manapun.

    TODO:
    - concat per grup.
    - X_morph shape akhir: [N, 250, 1] (tambah channel dim untuk Conv1D).
    - X_rr shape: [N, 3].
    - simpan ke data/processed/*.npz supaya fase berikut cepat load.
    """
    raise NotImplementedError
```

Jebakan

- **Jangan** shuffle lalu split ulang setelah gabung — split-nya SUDAH ditentukan by record. Shuffle boleh **di dalam** DS1 saat training (batching), tapi DS2 jangan diapa-apain.
- Pastikan tidak ada record DS1 nyasar ke DS2 (typo angka). Assert himpunan set(DS1) & set(DS2) == {}.

Definition of Done

- ☐ set(DS1), set(DS2), set(PACED) disjoint; totalnya 48.
- ☐ File .npz train & test tersimpan, shape ter-print & masuk akal.
- ☐ X_morph_train.shape[0] (jumlah beat latih) ≈ puluhan ribu.

Cara verifikasi

Cetak jumlah beat & rasio kelas di DS1 dan DS2 terpisah. Rasio boleh beda antara DS1/DS2 (mang begitu sifat inter-patient) — itu justru realistis.

FASE 4 — Arsitektur Model Hybrid

Tujuan

Bangun model dua-cabang: morfologi (CNN dari window) + ritme (Dense dari fitur RR) → concat → sigmoid.

Kenapa

Cabang morfologi menangkap **bentuk** detak (QRS lebar? ada P?); cabang ritme menangkap **timing** (prematur? irreguler?). Aritmia sering ketahuan dari timing yang morfologi saja gak lihat. Nambah cabang RR cuma nambah sedikit parameter (Dense terima 35 bukan 32 input) → total tetap ~6.000, INT8 < 20 KB.

Yang dibangun — src/model.py

Ini spec dari dokumen PA lo; rakit dengan **Functional API Keras** (wajib Functional karena dua input):

```
import tensorflow as tf
from tensorflow.keras import layers, Model
from config import WIN_LEN

def build_hybrid_model():
    """
    Cabang morfologi:
    Input (WIN_LEN, 1)
    Conv1D(16, 7, ReLU) → MaxPool
    Conv1D(32, 5, ReLU) → MaxPool
    Conv1D(32, 3, ReLU) → MaxPool
    GlobalAveragePooling1D → vektor 32-dim

    Cabang ritme:
    Input (3,) # RR_prev, RR_ratio, dRR

    Gabung:
    concat([morph_32, rr_3]) → 35-dim
    Dense(16, ReLU) → (Dropout opsional)
    Dense(1, sigmoid) → y_hat (>=0.5 → Aritmia)

    Returns: keras.Model dengan 2 input, 1 output.

    TODO: rakit pakai Functional API. Verifikasi model.count_params() ~ 6000.
    """
    raise NotImplementedError
```

Kenapa GAP bukan Flatten: Global Average Pooling meringkas tiap feature map jadi 1 angka → parameter Dense turun drastis dibanding Flatten, model tetap mungil & lebih tahan overfitting. Ini alasan model bisa < 20 KB.

Decision point

- **Dropout** di Dense 16: pakai atau tidak (mis. 0.3). Kalau dataset timpang & model kecil, dropout ringan bisa bantu. Catat pilihannya.
- **Loss:** binary_crossentropy. **Output activation:** sigmoid (sudah fixed).

Definition of Done

- ☐ model.summary() jalan, 2 input terlihat, output shape (None, 1).
- ☐ model.count_params() di kisaran ~6.000 (kalau jauh lebih besar, cek: mungkin gak sengaja pakai Flatten atau Dense kegedean).

Cara verifikasi

Feed dummy: model.predict([np.zeros((4,250,1)), np.zeros((4,3))]) → keluar shape (4,1), nilai di [0,1].

FASE 5 — Training + Penanganan Imbalance

Tujuan

Latih model **hanya di DS1**, dengan penanganan kelas timpang, hasilkan model_fp32.keras.

Kenapa

Distribusi MIT-BIH sangat timpang (Normal dominan). Tanpa penanganan, model belajar “tebak Normal terus” → accuracy tinggi tapi **recall Aritmia rendah** = gagal total untuk deteksi dini. Recall Aritmia adalah metrik yang paling lo pedulikan.

Yang dibangun — src/train.py

```
from sklearn.utils.class_weight import compute_class_weight
import numpy as np

def make_class_weights(y_train):
    """
    Hitung bobot inversely-proportional ke frekuensi kelas.
    TODO: compute_class_weight("balanced", classes=[0,1], y=y_train)
    → dict {0: w0, 1: w1}. Bobot kelas 1 (Aritmia) akan > 1.
    """
    raise NotImplementedError

def train(model, X_morph_tr, X_rr_tr, y_tr):
    """
    TODO:
    - compile(optimizer="adam", loss="binary_crossentropy",
              metrics=[Recall, Precision, AUC]).
    - fit([X_morph_tr, X_rr_tr], y_tr, class_weight=..., epochs=...,
          validation_split dari DS1 saja – JANGAN pakai DS2 sbg val).
    - EarlyStopping monitor val recall/AUC.
    - simpan model_fp32.keras.
    Lock np.random & tf.random seed = SEED sebelum training.
    """
    raise NotImplementedError
```

Decision point

- **Strategi imbalance:** class_weight (paling simpel, direkomendasikan dokumen lo) **atau** over-sampling kelas minoritas. Pilih satu, catat. Jangan dua-duanya sekaligus (bisa over-koreksi).
- **Validation set:** ambil dari **DS1** (mis. sisihkan beberapa record DS1 sebagai val, atau split beat DS1). **DS2 haram jadi validation** — DS2 hanya disentuh sekali di Fase 6 sebagai test final.
- **Threshold 0,5:** default. Nanti bisa diturunkan buat naikin sensitivity (dokumen lo bilang threshold kalibratable) — tapi kalibrasi threshold pakai kurva di DS1/val, bukan DS2.

Jebakan

- **Val = DS2** → bocor. Kalau lo tuning epoch/threshold berdasarkan DS2, DS2 gak murni lagi.
- **Seed gak di-lock** → hasil gak reproducible, pengujian minta ulang, angka beda.
- **Accuracy sebagai metrik utama** → menyesatkan di data timpang. Pantau **recall Aritmia + AUC**, bukan accuracy.

Definition of Done

- ☐ model_fp32.keras tersimpan.
- ☐ Kurva training (loss/recall vs epoch) tersimpan di artifacts/metrics/.
- ☐ Recall Aritmia di validation (dari DS1) sudah “sehat” (bukan ~0).

Cara verifikasi

Plot training history. Kalau val recall Aritmia stuck rendah sementara accuracy tinggi → class weight gak dipasang atau kurang kuat.

FASE 6 — Evaluasi Float32 di DS2

Tujuan

Ukur performa sesungguhnya di **DS2 (inter-patient, belum pernah dilihat)**: confusion matrix + accuracy/precision/recall/F1.

Kenapa

Ini **angka klaim ilmiah primer** PA lo. Karena DS2 pasiennya beda total dari DS1, angka ini jujur menggambarkan generalisasi.

Yang dibangun — src/evaluate.py

```
from sklearn.metrics import (confusion_matrix, classification_report,
                             precision_recall_fscore_support)

def evaluate_fp32(model, X_morph_te, X_rr_te, y_te, threshold=0.5):
    """
    TODO:
    - y_prob = model.predict([X_morph_te, X_rr_te]).
    - y_pred = (y_prob >= threshold).astype(int).
    - hitung confusion_matrix (label 0=Normal, 1=Aritmia).
    - hitung accuracy, precision, recall, F1 (fokus laporkan kelas Aritmia).
    - simpan confusion matrix (angka + heatmap PNG) & tabel metrik CSV.
    Returns: dict metrik.
    """
    raise NotImplementedError
```

Decision point

- **Agregasi metrik:** hitung di level **beat** (semua beat DS2 digabung). Ini standar & sebanding dgn literatur. (Catat kalau advisor mau level record.)
- **Kelas mana yang jadi “positive”:** Aritmia = positive (karena itu yang mau dideteksi). Recall Aritmia = seberapa banyak aritmia yang ketangkap; ini yang paling penting untuk early warning.

Jebakan

- **Threshold di-tuning pakai DS2** → bocor. Threshold difinalkan di Fase 5 (dari DS1/val). Di DS2 lo lapor apa adanya di threshold itu.
- Jangan cuma lapor accuracy. Data timpang bikin accuracy terlihat bagus meski recall Aritmia jelek.

Definition of Done

- ☐ Confusion matrix DS2 tersimpan (angka + PNG).
- ☐ Tabel accuracy/precision/recall/F1 (highlight recall Aritmia) tersimpan.
- ☐ Angka ini jadi baris “float32” di tabel delta Fase 7.

Cara verifikasi

Confusion matrix masuk akal: TN besar (Normal banyak & benar), TP > 0 dan recall Aritmia “layak” untuk sebuah model ~6k param. Bandingkan kasar dengan anchor Zambrano/Kim (binary ~97%) — lo gak harus setara, tapi kalau recall Aritmia ~0, ada bug.

FASE 7 — Kuantisasi INT8 + Tabel Delta

Tujuan

Kuantisasi model_fp32 → INT8 via PTQ, evaluasi ulang di **DS2 identik**, dan hitung **delta tiap metrik**. Verifikasi ukuran < 20 KB.

Kenapa

Device jalanin INT8, bukan float32. Klaim “dampak kuantisasi diukur, bukan diasumsikan” (Tabel 3.4 lo) butuh angka delta nyata. Kalau delta kecil (mis. recall turun < 1%), itu bukti kuat kelayakan.

Yang dibangun — src/quantize.py

```
import tensorflow as tf
import numpy as np

def representative_dataset_gen(X_morph_ds1, X_rr_ds1, n=300):
    """
    Generator sampel kalibrasi untuk PTQ.

    JEBAKAN #3 (dua-input): model lo punya DUA input. Generator HARUS
    yield list [window_sample, rr_sample], bukan cuma window.
    Kalau salah, converter error atau kuantisasi rentang RR ngaco.

    JEBAKAN #4 (cakupan kelas): sampel representatif WAJIB mencakup
    SEMUA kelas (Normal DAN Aritmia), bukan cuma Normal. Kalau cuma
    Normal, rentang aktivasi kelas minoritas gak terkalibrasi →
    INT8 rusak untuk Aritmia. Ambil sampel dari DS1 secara stratified.

    TODO: yield [x_morph[i:i+1].astype(f32), x_rr[i:i+1].astype(f32)]
           untuk n sampel stratified dari DS1.
    """
    raise NotImplementedError

def quantize_int8(keras_model_path, rep_gen):
    """
    TODO:
    - converter = tf.lite.TFLiteConverter.from_keras_model(...).
    - converter.optimizations = [tf.lite.Optimize.DEFAULT].
    - converter.representative_dataset = rep_gen.
    - target full-INT8:
      converter.target_spec.supported_ops =
        [tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
      converter.inference_input_type = tf.int8 # atau float32, catat pilihan
      converter.inference_output_type = tf.int8
    - convert, simpan model_int8.tflite, print ukuran byte.
    """
    raise NotImplementedError

def evaluate_int8(tflite_path, X_morph_te, X_rr_te, y_te):
    """
    Jalankan inferensi via tf.lite.Interpreter di seluruh DS2.
    TODO:
    - Interpreter(model_path).allocate_tensors().
    - untuk tiap sampel: set kedua input tensor (perhatikan urutan &
      scale/zero_point kalau input INT8), invoke, baca output, dequantize.
    - hitung metrik SAMA seperti Fase 6.
    Returns: dict metrik INT8.
    """
    raise NotImplementedError
```

Tabel delta (deliverable akhir)

Metrik	Float32 (DS2)	INT8 (DS2)	Delta
Accuracy	...	...	...
Precision (Aritmia)	...	...	...
Recall (Aritmia)	...	...	...
F1 (Aritmia)	...	...	...
Ukuran model	(n/a)	... KB	—

Decision point

- **Tipe input/output** INT8 penuh vs input float32: kalau device lo feed float lalu internal INT8, `inference_input_type=float32` lebih gampang. Kalau mau full-INT8 end-to-end, INT8. Catat & konsisten dgn rencana firmware.
- **Jumlah sampel representatif** (~100-500): lebih banyak = kalibrasi lebih stabil, tapi lebih lambat convert.

Jebakan

- Representative gen cuma satu input → converter gagal (JEBAKAN #3).
- Representative gen cuma kelas Normal → INT8 jeblok di Aritmia (JEBAKAN #4).
- Lupa dequantize output INT8 saat evaluasi → metrik ngawur.

Definition of Done

- ☐ `model_int8.tflite` tersimpan, ukuran ter-print < **20 KB**.
- ☐ Metrik INT8 di DS2 dihitung dengan cara identik Fase 6.
- ☐ **Tabel delta lengkap** tersimpan (CSV + versi buat dimasukkan ke laporan).

Cara verifikasi

Delta recall Aritmia float32→INT8 kecil (idealnya < 1-2%). Kalau anjlok besar → cek representative dataset (kemungkinan besar cakupan kelas atau dua-input salah).

FASE 8 — PoC Selesai: Checklist Final

PoC dinyatakan **selesai** kalau semua ini centang:

- ☐ **Reproducible:** clone repo → `pip install -r requirements.txt` → jalankan pipeline → angka sama (seed di-lock).
- ☐ **Preprocessing kausal** (sosfilt, bukan filtfilt) — golden reference siap diport ke C.
- ☐ **Split inter-patient** DS1/DS2 terverifikasi disjoint, paced dibuang.
- ☐ **Model hybrid** ~6.000 param, 2 input, summary bersih.
- ☐ **Metrik float32 DS2** ada (confusion matrix + recall Aritmia).
- ☐ **Model INT8** < 20 KB, metrik INT8 DS2 ada.
- ☐ **Tabel delta** float32 vs INT8 lengkap.
- ☐ **Semua decision point tercatat** (orde filter, definisi RR, mapping LBBB, strategi imbalance, tipe I/O INT8, threshold).

Kalau ini beres, lo bisa nulis di laporan: *“Kelayakan inferensi aritmia on-device via 1D-CNN hybrid terkuantisasi INT8 telah dibuktikan secara offline pada MIT-BIH inter-patient; degradasi akibat kuantisasi terukur sebesar Δ .”* — dan itu klaim yang **kuat dan tahan sidang**.

LAMPIRAN A — Urutan Eksekusi Cepat

Kalau lo mau peta satu layar:

Fase 0	io_mitdb.py	→ load_record() jalan, R-peak overlay pas
Fase 1	preprocessing.py	→ bandpass kausal + Pan-Tompkins + window 250 + z-score
Fase 2	features_rr.py	→ mapping AAMI + label biner + fitur RR (pakai anotasi)
Fase 3	dataset.py	→ DS1/DS2 disjoint, .npz tersimpan
Fase 4	model.py	→ hybrid ~6k param
Fase 5	train.py	→ latih di DS1, class_weight, seed lock
Fase 6	evaluate.py	→ metrik float32 di DS2
Fase 7	quantize.py	→ INT8 <20KB + tabel delta
Fase 8	checklist	→ PoC done

LAMPIRAN B — Peta “Why A not B” (amunisi sidang)

Tiap keputusan punya alternatif yang ditolak. Hafal rasionalnya:

Keputusan	Alternatif ditolak	Rasional singkat
Binary (Normal/Aritmia)	5-kelas AAMI	Inter-patient bikin F & Q nyaris nol recall → metrik menyesatkan
Inter-patient split	Intra-patient / random	Random bocor per-pasien → akurasi palsu tinggi
Filter kausal (sosfilt)	filtfilt zero-phase	filtfilt mustahil real-time di MCU → train/deploy mismatch
Segmentasi pakai anotasi	pakai Pan-Tompkins	Jaga jalur model murni; deteksi diukur terpisah
z-score per window	normalisasi global	Kebal beda amplitudo + nol kebocoran statistik
GAP	Flatten	Param jauh lebih kecil → muat MCU, anti-overfit
class_weight	abaikan imbalance	Tanpa itu recall Aritmia ~0 = gagal deteksi dini
Rep. dataset semua kelas	cuma Normal	Kalibrasi INT8 kelas minoritas rusak kalau cuma Normal

LAMPIRAN C — Yang Menunggu Hardware (di luar PoC)

Setelah PoC beres & alat datang, jalur berikutnya (bukan bagian dokumen ini):

- Export .tflite → array C (xxd -i atau .h), integrasi TFLite Micro di ESP32-S3.
 - Port golden reference preprocessing ke C (biquad + Pan-Tompkins streaming) — **harus identik** dengan Python.
 - Benchmark on-device: latensi per-detak (timestamp MCU), tensor arena RAM, ukuran flash, daya via INA219.
 - Pipeline resilience: PSRAM ring buffer, MQTT QoS-1 offloading, ThingsBoard dedup.
 - Uji fungsional relawan sehat (jalur pipeline, bukan klaim klinis).
-

Dokumen ini menggantikan sketsa fase ringkas sebelumnya. Semua konstanta mengikuti desain di BAB 3 PA. Cross-check daftar DS1/DS2 ke de Chazal 2004 sebelum eksekusi Fase 3.